Emergency Training Simulator POC

Submission Packet

Roadmap

Project roadmap

Source: docs/roadmap.md

Prepared for review, discussion, and project handoff.

Markdown stays editable. This PDF is the clean reading copy.

Boundary: this is still a POC, not operational or clinical guidance.

Roadmap

Reviewed October 28, 2026. Current status supplies the evidence behind this roadmap. Full-system gap map compares the implemented POC with the long-term vision.

Implemented foundation

- Deterministic sessions, two toy scenarios, ordered WebSocket contracts, explicit validation, JSONL replay/review and local guardrails.

- React studio, hosted ARC room planner, validated seeded layouts, persistent jobs/history, exact-input reuse and recoverable Trash.

- Generated textured chair, reviewed Blender repair and 19-piece collision baseline; user-confirmed Blender room assembly and Unreal 5.8 chair/room/Chaos demos.

- Separate browser Rapier primitive proof and legacy fixture authoring/revision approval.
- Furniture variant library, optional reference/request/result ZIP route, actual GLB viewer.
- Description-based asset automation, SSH/Slurm dispatch/recovery and validated retrieval.
- Blender MCP preparation, CPU convex decomposition, editable .blend, preview, FBX/material package and a generic Unreal import helper.

- Recording controls, saved-frame encoding/retry, artifact downloads and CI.
The last three automation/recording areas have partial runtime evidence; see the next checks below. A completed implementation list is not a claim of live acceptance.

Next acceptance checkpoints

1. One-time SSH/agent/host trust and ARC checkout update. Evidence: the app worker check succeeds without passwords or weakened host verification.

2. First automated charcoal couch. Evidence: actual Slurm ID, successful image/shape/paint stages, validated lineage and inspected real model.

3. Blender preparation of that actual couch. Evidence: inspected shape/materials, reported dimensions, preserved source, collision fit and saved editable result.

4. Generic Unreal furniture import. Evidence: actual engine import, material wiring, size/hull/cooking checks and bounded contact inspection.

5. First live movement recording. Evidence: actual engine frames/MP4, matching observations and useful framing.

6. Mixed-furniture rooms. Evidence: planner, reviewed asset resolver, footprints and Blender/Unreal bindings agree on IDs/dimensions without silent substitutions.

Reuse existing runtimes and accepted chair demos. Do not rebuild model environments or rerun generation merely to refresh documentation. Resume uncertain submissions using their saved dispatch rather than creating another GPU job.

Complete-environment direction

The intended user flow is a room/scenario description that resolves reviewed assets, generates missing designs explicitly, places furniture using actual footprints, saves an

editable Blender environment and assembles the same scene in Unreal.

Individual asset automation is now implemented. The current room lane still uses the reviewed chair. The missing bridge is coordinated mixed-asset planning and binding, followed by end-to-end visual/engine acceptance. Scenario semantics, navigation, smoke/fire behavior and trainee interactions need their own contracts and experiments; a generated room is not automatically a training simulation.

Later research and deployment

- Measure queue/inference/cleanup/import time and review effort across designs.
- Improve model quality, provenance review, collision fit and asset reuse/versioning.
- Connect instructor-reviewed scenario content to runtime behavior with explicit authority.
- Expand physics measurements and independent validation before claiming realism.
- Define sensing, confidence/privacy, mannequin and projection interfaces.
- Validate real-time adaptation and domain-reviewed training content separately.
- Plan CS Launch hosting, institution-approved ARC service access, authentication, roles, retention and multi-site operation before moving beyond loopback.

Build strategy

One solo-developer checkpoint should produce a concrete, reviewable result and preserve earlier working demos. Prefer measured evidence to completion percentages or calendar promises. The backend owns accepted state, validation and review; generation proposes artifacts and engines report observations.